

# Node.js Interview Questions & Answers

Basic • Intermediate • Advanced | 30 Questions | With Code Examples

10  
Basic

8  
Intermediate

12  
Advanced

## BASIC QUESTIONS

**Q 1.** Node.js kya hai aur yeh kyun use kiya jata hai?

Basic

- A** Node.js ek open-source, cross-platform JavaScript runtime environment hai jo Chrome ke V8 engine par built hai.
- Yeh server-side JavaScript execute karne ke liye use hota hai. Non-blocking, event-driven architecture ki wajah se yeh highly scalable aur fast applications banane ke liye ideal hai, especially jab bahut saare concurrent connections handle karne ho.

**Co** `const http = require('http'); http.createServer((req, res) => { res.write("Hello Node.js!");`  
**de:** `res.end(); }).listen(3000);`

**Q 2.** Node.js event loop kaise kaam karta hai?

Basic

- A** Event loop Node.js ka core mechanism hai jo non-blocking I/O operations ko handle karta hai. Jab koi async operation hoti hai (jaise file read ya DB query), woh callback queue mein jayegi. Event loop continuously check karta hai ki call stack empty hai ya nahi, agar hai toh callback queue se task uthata hai aur execute karta hai.

**Co** `console.log("Start"); setTimeout(() => console.log("Timeout"), 0); Promise.resolve().then(()`  
**de:** `=> console.log("Promise")); console.log("End"); // Output: Start -> End -> Promise -> Timeout`

**Q 3.** `require()` aur `import` mein kya difference hai?

Basic

- A** `require()` CommonJS module system ka hissa hai jo Node.js mein default hota hai. Yeh synchronous hai aur runtime par modules load karta hai. `import` ES6 module syntax hai jo static aur asynchronous hota hai. Modern Node.js mein dono support hote hain.

**Co** `// CommonJS const fs = require('fs'); // ES Modules (.mjs file) import fs from 'fs'; import {`  
**de:** `readFile } from 'fs/promises';`

**Q 4. npm kya hai? package.json ki kya importance hai?**

Basic

**A** npm (Node Package Manager) duniya ka sabse bada software registry hai. Isse libraries/packages install, share aur manage kiye ja sakte hain. package.json ek configuration file hai jisme project ki saari information hoti hai — name, version, dependencies, scripts, etc.

```
Co // package.json { "name": "my-app", "version": "1.0.0", "scripts": { "start": "node
de: index.js", "dev": "nodemon index.js" }, "dependencies": { "express": "^4.18.2" } }
```

**Q 5. Callback kya hota hai? Callback hell kya hai?**

Basic

**A** Callback ek function hai jo doosre function ko argument ke roop mein pass kiya jata hai aur baad mein execute hota hai. Callback hell tab hoti hai jab multiple nested callbacks ek doosre ke andar hoti hain, jisse code padhna mushkil ho jata hai. Ise "Pyramid of Doom" bhi kehte hain.

```
Co // Callback Hell (bura tarika) getUser(id, (user) => { getOrders(user, (orders) => {
de: getProduct(orders[0], (product) => { // aur andar... }); }); });
```

**Q 6. Synchronous aur Asynchronous programming mein kya fark hai?**

Basic

**A** Synchronous programming mein code ek line ke baad doosri line execute hoti hai. Asynchronous programming mein time-consuming tasks background mein chalte hain aur code block nahi hota. Node.js asynchronous model use karta hai jisse high concurrency handle hoti hai.

```
Co // Synchronous (blocks karta hai) const data = fs.readFileSync("file.txt");
de: console.log(data); // Asynchronous (non-blocking) fs.readFile("file.txt", (err, data) => {
    console.log(data); }); console.log("Yeh pehle print hoga");
```

**Q 7. Streams kya hote hain Node.js mein?**

Basic

**A** Streams data ko chunks mein process karte hain, poora data memory mein load karne ki zaroorat nahi hoti. 4 types hote hain: Readable, Writable, Duplex, Transform. Bade files ya network requests ke liye ideal hain.

```
Co const fs = require('fs'); const readable = fs.createReadStream("input.txt"); const writable
de: = fs.createWriteStream("output.txt"); readable.pipe(writable);
```

Q  
8.

Buffer kya hota hai Node.js mein?

Basic

**A** Buffer ek temporary memory storage hai jo binary data store karta hai. Jab aap binary data (images, files, network packets) ke saath kaam karte ho toh Buffer use hota hai. Yeh fixed-size raw memory allocation hai jo Uint8Array class par based hai.

**Co** `const buf = Buffer.from("Hello", "utf8"); console.log(buf); // <Buffer 48 65 6c 6c 6f>`  
**de:** `console.log(buf.toString()); // Hello` `const buf2 = Buffer.alloc(10); // 10 bytes ka empty buffer`

## INTERMEDIATE QUESTIONS

Q  
9.

Promise kya hai? async/await kaise kaam karta hai?

Intermediate

**A** Promise ek object hai jo future mein complete hone wali async operation ko represent karta hai. 3 states hote hain: pending, fulfilled, rejected. async/await Promise ke upar ek syntactic sugar hai jo asynchronous code ko synchronous jaisa likhne deta hai.

**Co** `// async/await async function getUser() { try { const res = await fetch("/api/user"); const`  
**de:** `user = await res.json(); console.log(user); } catch(err) { console.error(err); } }`

Q  
10

Express.js kya hai? Middleware kaise kaam karta hai?

Intermediate

**A** Express.js Node.js ka sabse popular web framework hai. Middleware functions woh functions hain jo request-response cycle mein req, res aur next middleware tak access rakhte hain. Authentication, logging, error handling sab middleware se kiya jata hai.

**Co** `const express = require('express'); const app = express(); // Custom middleware`  
**de:** `app.use((req, res, next) => { console.log(`${req.method} ${req.url}`); next(); // agla middleware/route chalo }); app.get('/', (req, res) => { res.json({ message: "Hello!" }); }); app.listen(3000);`

Q  
11

process.nextTick() aur setImmediate() mein kya fark hai?

Intermediate

**A** process.nextTick() current operation complete hone ke baad, event loop ke kisi bhi phase se pehle execute hota hai — highest priority callback. setImmediate() event loop ke "check" phase mein execute hota hai, I/O callbacks ke baad.

**Co** `setImmediate(() => console.log("setImmediate")); process.nextTick(() =>`  
**de:** `console.log("nextTick")); Promise.resolve().then(() => console.log("Promise")); // Output order: // Promise -> nextTick -> setImmediate`

**Q****12 EventEmitter kya hai? Custom events kaise banate hain?****Intermediate**

.

**A** EventEmitter Node.js mein events publish-subscribe pattern implement karta hai. Aap custom events emit kar sakte ho aur unhe listen kar sakte ho. Node.js ke bahut saare built-in modules EventEmitter se inherit karte hain.

```
Co const { EventEmitter } = require('events'); const emitter = new EventEmitter();  
de: emitter.on("data", (msg) => { console.log("Data mila:", msg); }); emitter.emit("data",  
    "Hello World!");
```

**Q****13 Cluster module kya hai aur kyun use karte hain?****Intermediate**

.

**A** Node.js single-threaded hai isliye ek CPU core ka use karta hai. Cluster module se multiple child processes bana sakte ho jo same server port share karte hain. Multi-core systems ka poora faida milta hai aur performance improve hoti hai.

```
Co const cluster = require('cluster'); const os = require('os'); if (cluster.isPrimary) { const  
de: cpus = os.cpus().length; for (let i = 0; i < cpus; i++) { cluster.fork(); } } else {  
    require('./server'); // Worker process }
```

**Q****14 JWT authentication Node.js mein kaise implement karte hain?****Intermediate**

.

**A** JWT (JSON Web Token) ek stateless authentication method hai. Server token generate karta hai jo client store karta hai. Har request mein token bheja jata hai, server verify karta hai. Token mein header, payload aur signature hoti hai.

```
Co const jwt = require('jsonwebtoken'); // Token generate karo const token = jwt.sign( { userId:  
de: 123, role: "admin" }, process.env.JWT_SECRET, { expiresIn: "24h" } ); // Middleware: Token  
verify karo function authMiddleware(req, res, next) { const token =  
    req.headers.authorization?.split(" ")[1]; try { req.user = jwt.verify(token,  
        process.env.JWT_SECRET); next(); } catch { res.status(401).json({ error: "Invalid token" });  
    } }
```

**Q****15 CORS kya hai? Node.js mein kaise handle karte hain?****Intermediate**

.

**A** CORS (Cross-Origin Resource Sharing) ek security feature hai jo browser mein built-in hai. Jab ek domain se doosre domain par request jaati hai toh browser CORS check karta hai. Express mein cors package se easily handle kar sakte hain.

```
Co const cors = require('cors'); // Specific origins ke liye app.use(cors({ origin:  
de: ['https://mysite.com', 'https://app.mysite.com'], methods: ['GET', 'POST', 'PUT', 'DELETE'],  
    allowedHeaders: ['Content-Type', 'Authorization'] }));
```

**Q 16 Environment variables kaise use karte hain?**

Intermediate

**A** Environment variables sensitive information (API keys, DB passwords) store karne ke liye use hoti hain. Code mein hardcode nahi karni chahiye. dotenv package .env file se variables load karta hai.

**Co** // .env file DB\_HOST=localhost JWT\_SECRET=mysupersecret PORT=3000 // index.js  
**de:** require('dotenv').config(); const port = process.env.PORT || 3000; app.listen(port);

**Q 17 Rate limiting kya hai aur kyun zaruri hai?**

Intermediate

**A** Rate limiting ek technique hai jo server ko abuse se bachati hai. Ek client kitni baar ek time mein request kar sakta hai yeh limit karta hai. DDoS attacks, brute force aur API abuse se protection milti hai.

**Co** const rateLimit = require('express-rate-limit'); const limiter = rateLimit({ windowMs: 15 \* 60 \* 1000, // 15 minutes max: 100, message: "Bahut saari requests! Thodi der baad try karo." }); app.use(limiter);

**Q 18 Memory leak kaise detect aur fix karte hain?**

Intermediate

**A** Memory leaks tab hoti hain jab memory allocate hoti hai lekin garbage collector isse free nahi kar paata. Common causes: global variables, event listeners jo remove nahi hue. Detection ke liye --inspect flag, Chrome DevTools use kar sakte hain.

**Co** // Galat - Memory leak const cache = {}; app.get('/data', (req, res) => { cache[Date.now()] = new Array(10000).fill("data"); // grows forever! }); // Sahi - Bounded cache with size limit  
**de:** const MAX = 100; const keys = []; // Remove old entries when limit reached

## ADVANCED QUESTIONS

**Q 19 Worker Threads kya hain? Cluster se kaise alag hain?**

Advanced

**A** Worker Threads ek hi process mein multiple threads banate hain jo actually parallel mein chalte hain aur memory share kar sakte hain. CPU-intensive tasks (image processing, ML) ke liye ideal hain. Cluster alag processes banata hai jo load balancing ke liye better hain.

**Co** const { Worker, isMainThread, parentPort } = require('worker\_threads'); if (isMainThread) {  
**de:** const worker = new Worker(\_\_filename); worker.on('message', (result) => console.log("Result:", result)); worker.postMessage({ n: 40 }); } else {  
parentPort.on('message', ({ n }) => { parentPort.postMessage(fibonacci(n)); // Heavy computation }); }

**Q 20 Node.js mein caching strategies kya hain?**

Advanced

**A** In-memory caching (node-cache, LRU cache) fastest hai lekin restarts par clear hoti hai. Redis distributed caching ke liye best hai — multiple instances ke beech share hoti hai. HTTP caching browser aur CDN level par kaam karta hai.

```
Co const NodeCache = require('node-cache'); const cache = new NodeCache({ stdTTL: 300 }); // 5
de: min TTL async function getUser(id) { const cached = cache.get(`user:${id}`); if (cached)
    return cached; const user = await db.findUser(id); cache.set(`user:${id}`, user); return
    user; }
```

**Q 21 Database connection pooling kya hai?**

Advanced

**A** Connection pooling mein database connections ko pre-create karke ek pool mein rakha jata hai. Har request ke liye naya connection banana expensive hai. Pool se available connection liya jata hai, use karke wapas rakha jata hai. Latency kam hoti hai.

```
Co const { Pool } = require('pg'); const pool = new Pool({ max: 20, // max connections
de: idleTimeoutMillis: 30000, connectionTimeoutMillis: 2000 }); async function query(sql,
    params) { const client = await pool.connect(); try { return await client.query(sql, params);
    } finally { client.release(); // pool mein wapas } }
```

**Q 22 GraphQL vs REST API — kaun better hai?**

Advanced

**A** REST mein har resource ke liye alag endpoint hota hai. GraphQL mein client specify karta hai ki exactly kaunsa data chahiye — over-fetching aur under-fetching ki problem solve hoti hai. Complex data requirements ke liye GraphQL, simple CRUD ke liye REST sufficient hai.

```
Co // REST - multiple calls zaroori GET /users/1 GET /users/1/posts GET /users/1/friends //
de: GraphQL - ek call mein sab query { user(id: 1) { name posts { title } friends { name } } }
```

**Q 23 Microservices mein Node.js ka role kya hai?**

Advanced

**A** Node.js microservices ke liye ideal hai: lightweight, fast startup, non-blocking I/O. Services HTTP/REST, gRPC, ya message queues (RabbitMQ, Kafka) se communicate karte hain. Har service independent deploy hoti hai.

```
Co // API Gateway pattern gateway.use('/users', createProxyMiddleware({ target:
de: 'http://user-service:3001' })); // Message Queue const channel = await conn.createChannel();
    await channel.sendToQueue( 'orders', Buffer.from(JSON.stringify(order)) );
```

**Q 24 Node.js ko Docker mein kaise containerize karte hain?**

Advanced

**A** Docker containers applications ko unki dependencies ke saath package karte hain. Dockerfile image define karta hai. docker-compose multiple services ek saath manage karta hai.

```
Co # Dockerfile FROM node:18-alpine WORKDIR /app COPY package*.json ./ RUN npm ci
de: --only=production COPY . . EXPOSE 3000 CMD ["node", "index.js"] # docker-compose.yml
services: app: build: . ports: ["3000:3000"] postgres: image: postgres:14
```

**Q 25 Error handling best practices kya hain?**

Advanced

**A** Errors do types ke hote hain: Operational (expected) aur Programmer errors (bugs). Centralized error handler middleware use karo. Unhandled rejections ko catch karo aur graceful shutdown karo.

```
Co // Centralized error handler app.use((err, req, res, next) => { console.error(err.stack);
de: res.status(err.status || 500).json({ error: err.message }); }); // Unhandled rejections
process.on('unhandledRejection', (reason) => { console.error('Unhandled:', reason);
process.exit(1); });
```

**Q 26 PM2 kya hai aur kyun use karte hain?**

Advanced

**A** PM2 (Process Manager 2) ek production-ready process manager hai. Application ko crash hone par automatically restart karta hai, cluster mode mein chala sakta hai, logs manage karta hai. Development mein nodemon, production mein PM2.

```
Co # Install npm install -g pm2 # Cluster mode (all CPU cores) pm2 start index.js -i max #
de: ecosystem.config.js module.exports = { apps: [{ name: "my-app", script: "index.js",
instances: "max", env_production: { NODE_ENV: "production" } }] };
```

**Q 27 WebSocket Node.js mein kaise implement karte hain?**

Advanced

**A** WebSocket ek protocol hai jo client aur server ke beech full-duplex communication provide karta hai. Server bhi client ko message bhej sakta hai. Real-time applications (chat, live notifications, games) ke liye use hota hai.

```
Co const { Server } = require('socket.io'); const io = new Server(httpServer);
de: io.on("connection", (socket) => { socket.on("message", (data) => { io.emit("message", {
user: socket.id, text: data }); }); socket.on("disconnect", () => { console.log("User
disconnected"); }); });
```

**Q 28 Node.js mein testing kaise karte hain?**

Advanced

**A** Unit tests individual functions test karte hain, Integration tests multiple components mein, E2E tests पूरी user journey. Jest ya Mocha popular test frameworks hain. Supertest HTTP endpoints test karne ke liye use hota hai.

**Co** `const request = require('supertest'); const app = require('./app'); describe('GET /users/:id', () => { test('Valid user return karna chahiye', async () => { const res = await request(app).get('/users/1'); expect(res.statusCode).toBe(200); expect(res.body).toHaveProperty('name'); }); test('Invalid ID par 404 aana chahiye', async () => { const res = await request(app).get('/users/999'); expect(res.statusCode).toBe(404); }); });`

**Q 29 Performance optimization techniques kya hain?**

Advanced

**A** Async operations use karo, caching implement karo, database queries optimize karo (indexes), compression use karo (gzip), connection pooling karo, CDN use karo static files ke liye. Profiling se bottlenecks identify karo.

**Co** `// Compression const compression = require('compression'); app.use(compression()); // Parallel async operations async function getDashboard(userId) { const [user, orders, notifications] = await Promise.all([ getUser(userId), getOrders(userId), getNotifications(userId) ]); return { user, orders, notifications }; }`

**Q 30 Security best practices Node.js mein kya hain?**

Advanced

**A** Input validate karo (joi/zod), SQL injection se bachao (parameterized queries), helmet.js se HTTP headers secure karo, rate limiting lagao, dependencies regularly update karo (npm audit), environment variables mein secrets rakho, HTTPS use karo.

**Co** `const helmet = require('helmet'); const { z } = require('zod'); app.use(helmet()); // Security headers // Input validation const userSchema = z.object({ email: z.string().email(), password: z.string().min(8).max(100) }); app.post('/register', async (req, res) => { try { const data = userSchema.parse(req.body); // Safe data use karo } catch (err) { res.status(400).json({ error: "Invalid input" }); } });`